

ProductDAOTest.java

```
package com.harisudhan.harisudhanmart.dao;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertTrue;

import com.harisudhan.harisudhanmart.model.Product;
import com.harisudhan.harisudhanmart.model.User;
import com.zaxxer.hikari.HikariConfig;
import com.zaxxer.hikari.HikariDataSource;
import java.io.InputStream;
import java.math.BigDecimal;
import java.sql.Connection;
import java.sql.Statement;
import org.junit.jupiter.api.AfterAll;
import org.junit.jupiter.api.BeforeAll;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

/**
 * DAO tests run against an embedded H2 instance, per Section 9 (jdbc:h2:mem:test;DB_CLOSE_DELAY=-1),
 * initialized from schema.sql before each test.
 */
class ProductDAOTest {

    private static HikariDataSource dataSource;
    private ProductDAO productDAO;
    private UserDAO userDAO;

    @BeforeAll
    static void setUpPool() {
        HikariConfig config = new HikariConfig();
        config.setJdbcUrl("jdbc:h2:mem:test;DB_CLOSE_DELAY=-1");
        config.setUsername("sa");
        config.setPassword("");
        config.setDriverClassName("org.h2.Driver");
        dataSource = new HikariDataSource(config);
    }

    @AfterAll
    static void tearDownPool() {
        dataSource.close();
    }

    @BeforeEach
    void resetSchema() throws Exception {
        try (Connection conn = dataSource.getConnection();
            InputStream in = getClass().getClassLoader().getResourceAsStream("schema.sql")) {
            String sql = new String(in.readAllBytes());
            try (Statement stmt = conn.createStatement()) {
                stmt.execute("DROP ALL OBJECTS");
                for (String statement : sql.split(";")) {
                    String trimmed = statement.trim();
                    if (!trimmed.isEmpty()) {
                        stmt.execute(trimmed);
                    }
                }
            }
        }
        productDAO = new ProductDAOImpl(dataSource);
        userDAO = new UserDAOImpl(dataSource);
    }

    @Test
```

```

void createAndFindProduct_roundTrips() throws Exception {
    User seller = new User();
    seller.setName("Seller One");
    seller.setEmail("seller1@test.com");
    seller.setPasswordHash("hash");
    seller.setRole(User.Role.SELLER);
    seller = userDAO.create(seller);

    Product product = new Product();
    product.setSellerId(seller.getId());
    product.setName("Test Widget");
    product.setDescription("A widget for testing");
    product.setPrice(new BigDecimal("19.99"));
    product.setStockQty(10);
    product.setCategory("Test");
    product.setImageUrl("http://example.com/widget.jpg");

    Product created = productDAO.create(product);
    assertTrue(created.getId() > 0);

    Product found = productDAO.findById(created.getId()).orElseThrow();
    assertEquals("Test Widget", found.getName());
    assertEquals(0, new BigDecimal("19.99").compareTo(found.getPrice()));
}

@Test
void search_filtersByKeyword() throws Exception {
    User seller = new User();
    seller.setName("Seller Two");
    seller.setEmail("seller2@test.com");
    seller.setPasswordHash("hash");
    seller.setRole(User.Role.SELLER);
    seller = userDAO.create(seller);

    Product p1 = new Product();
    p1.setSellerId(seller.getId());
    p1.setName("Blue Mug");
    p1.setPrice(new BigDecimal("5.00"));
    p1.setStockQty(3);
    p1.setCategory("Kitchen");
    productDAO.create(p1);

    Product p2 = new Product();
    p2.setSellerId(seller.getId());
    p2.setName("Red Notebook");
    p2.setPrice(new BigDecimal("3.00"));
    p2.setStockQty(20);
    p2.setCategory("Stationery");
    productDAO.create(p2);

    assertEquals(1, productDAO.search("mug", null).size());
    assertEquals(1, productDAO.search(null, "Stationery").size());
}
}

```